Mana Forge

Didactic planning document for a developer coming from a PHP 5 / FTP style background

Revision 3: plain-language architecture, controlled registration, Google reCAPTCHA, shareable MTG deck views, and transactional email delivery

What this document is trying to do
This version explains the project in language that should feel familiar to someone used to classic PHP sites uploaded by FTP.
Whenever a modern term appears, the document translates it into something practical: what it is, why it exists, and what part of the old mental model it replaces.
It also adds a concrete email plan for account activation, password recovery, approval notices, and other messages the system must send.

Short project sentence

Mana Forge is a web application for building playtest-only Magic: The Gathering proxy sheets, saving projects online, sharing read-only deck views by link, and exporting printable PDFs.

Architecture in one line

GitHub Pages serves the site files, a Cloudflare Worker behaves like the backend script endpoint, Cloudflare D1 stores the data, Google reCAPTCHA reduces bot registrations, and Resend sends account and recovery emails.

Prepared for implementation planning

1. Read this like a translation guide

This project uses modern names for pieces of web infrastructure, but the practical behavior can be mapped to older habits that many PHP developers already understand.

Modern term	A more relatable explanation	How to picture it if you used PHP hosting
Static site	A set of already-generated HTML, CSS, and JavaScript files. No page code runs on the host when the user opens the page.	Think “files uploaded to hosting by FTP”, except generated by a build step instead of being handwritten page by page.
Frontend	The part that runs in the browser.	What the user sees after the PHP page is already delivered.
Backend	The trusted code that can read secrets, write to the database, and decide whether a request is allowed.	What used to be your PHP scripts on the server.
API endpoint	A URL that accepts structured requests such as login, save project, or reset password.	A modern replacement for “ajax.php?action=saveDeck”.
Cloudflare Worker	A small backend program hosted by Cloudflare. It answers HTTP requests without you renting a VPS.	A hosted script endpoint, like a tiny always-on PHP backend, but not running under Apache or PHP-FPM.
D1 database	A managed SQL database connected to the Worker.	The project database that your old PHP app would open with PDO or mysqli.
Build	The step that generates the final files that will be published.	Comparable to compiling assets and producing the final folder that would be uploaded to public_html.
Deploy	Publishing the built files or backend code to the real internet environment.	The moment you upload files or push a release live.
Secret / environment variable	A value that exists only on the backend, such as API keys.	What should never be hardcoded into the PHP page you send to the browser.
Session cookie	A browser cookie proving the user is logged in.	Same idea as PHP session cookies, but managed by the Worker.
Query string	The “?something=value” part	Exactly the same thing you

Modern term	A more relatable explanation	How to picture it if you used PHP hosting
	of the URL.	used in PHP via $_GET.
Compressed share link	A URL that packs deck data into the link itself, then the browser unpacks it.	Like passing serialized data in the URL, except compressed so the link stays reasonably short.
Transactional email	System emails triggered by actions such as account activation or password reset.	Your app-generated emails, not a newsletter or marketing blast.

2. What we are building in plain language

Mana Forge is not meant to be an “official-looking card factory”. It is a playtest workshop tool. The user imports or types a deck, chooses a simple proxy style, previews printable sheets, and exports a PDF meant for private testing at home.

The application also has accounts, because saved projects, moderation, and controlled sharing require us to know who owns what.

The user-facing features are:

- Create and save proxy projects.
- Paste or edit decklists.
- Preview simplified proxy card layouts.
- Generate printable PDF sheets.
- Open a deck viewer through a shareable link.
- Request an account, recover a password, and receive system emails when needed.

3. The architecture explained without buzzwords

Think of the system as two separate things working together:

1. The published site files: these are the pages, scripts, and styles that the browser downloads from GitHub Pages.
2. The trusted service: this is the Cloudflare Worker, which behaves like the backend and talks to the database and email provider.

The easiest mental picture
GitHub Pages is the “public_html” folder on the internet.
The Worker is the server-side script layer that you do NOT upload by FTP into the same folder.
D1 is the SQL database behind that script layer.
Resend is the external mail service used when the system must send real emails that arrive reliably.

Google reCAPTCHA is the bot check placed on the public registration form.

Browser request flow

Step	What happens	Why it matters
1	The browser opens the site on GitHub Pages and downloads static files.	Fast, cheap hosting for the visible part of the app.
2	When the user logs in, saves a project, or asks for a reset, the browser sends a fetch request to the Worker.	The browser itself must never receive database credentials or email API keys.
3	The Worker validates the request, checks permissions, and reads or writes D1.	This is the trust boundary of the system.
4	If the action requires email, the Worker calls Resend using a secret API key.	Only the backend can safely send trusted system emails.
5	The Worker returns JSON to the browser, and the page updates the interface.	This is the modern equivalent of an AJAX response.

4. Why Next.js is still acceptable even though the site is static

A static site does not mean “simple HTML only”. It means the published host is serving already-built files, not executing request-time page code. Next.js is still useful here because it organizes the interface, routing, and component structure during development, then exports the finished files for GitHub Pages.

Relatable version: we are using a modern site-building framework to prepare the files, but the final host behaves more like old-school file hosting than like a PHP application server.

Implications of this decision:

- No PHP-like page code runs on GitHub Pages.
- No database connection exists inside the published page files.
- Every trusted action must go through the Worker.
- Share pages must be designed so they still work as regular static routes plus query-string data.

5. Main functional areas of the application

Area	What the user can do	Special notes
Projects	Create, rename, duplicate,	Owned by a logged-in user.

Area	What the user can do	Special notes
	archive, and delete proxy projects.	
Deck editing	Paste decklists such as “4 Lightning Bolt”, reorder rows, change counts, and edit simplified text fields.	Should stay easy for bulk edits.
Proxy styles	Pick minimal visual templates meant for playtesting, not for fake authenticity.	Every card face must visibly say PROXY or PLAYTEST.
Print setup	Choose page size, grid, cut marks, grayscale, and footer text.	These become saved print profiles.
PDF export	Generate printable sheets from the current deck.	The PDF build can stay in the browser with pdf-lib.
Sharing	Generate a read-only link that opens a deck visualization page.	Editing never happens from the share link.
Administration	Approve registrations, suspend users, and review share records.	Visible only to admins.

6. Registration and account control - revised to be more practical

Because registration is moderated, the cleanest design is to avoid collecting a permanent password before approval. That keeps the “pending request” stage simpler and avoids storing credentials for accounts that may never be activated.

Recommended account flow:

3. A visitor opens the registration request form.
4. The visitor enters display name, email address, optional note, and completes Google reCAPTCHA.
5. The Worker verifies the reCAPTCHA token with Google.
6. If valid, the Worker stores a pending registration request in D1.
7. The system sends a “request received” email.
8. An administrator approves or rejects the request from the admin area.
9. If approved, the system sends an activation email with a one-time link.
10. The user opens the activation link, sets a password, and confirms the email address in the same flow.
11. After activation, the user can log in and use the application.

Why this is better than asking for a password in the first form

We do not keep dormant passwords for people who may be rejected or may never return.
The approval email naturally becomes the first verified contact point.
The activation page can combine “verify email” and “create password”, which is easier for users and easier to explain.

7. reCAPTCHA in practical terms

reCAPTCHA is only for the public request form and any other public form that is likely to attract bots. It is not a login system and it is not a user identity system.

The old-school way to think about it: it is the modern version of adding a “prove you are not an automated script” gate before your form actually reaches the server logic.

Recommended version

Use Google reCAPTCHA v2 Checkbox as the default for registration requests. It is easier to explain, easier to test, and more visible to administrators than a score-only invisible system.

Implementation notes

- The frontend loads Google’s reCAPTCHA widget and obtains a token when the user completes the challenge.
- The browser sends that token to the Worker together with the registration form.
- The Worker sends the token to Google’s verification endpoint.
- Only if Google says the token is valid do we store the request.
- The reCAPTCHA secret key stays only in Worker secrets, never in the browser code.

8. Share links, explained for someone used to query strings

The project needs a way to share a deck visualization page by URL. There are two modes, and both should exist because each solves a different problem.

Mode	How it works	When to use it
Portable URL	The deck data is packed into the query string itself, for example ?deck=<compressed-data>. The viewer page unpacks it in the browser.	Best when the deck is small enough and we want a self-contained link with no extra database lookup.
Short server-backed URL	The browser sends the deck payload to the Worker, the Worker stores it in D1 and returns a short slug such as ?share=ABC123.	Best when the deck state is too big for a practical URL, or when we want expiration or moderation.

Relatable version: mode one is like stuffing a compressed parameter into $_GET. Mode two is like saving the payload in a table and passing only the record code in the URL.

Important safety rule

A share link opens only a read-only deck viewer. It must never expose edit screens, admin data, private notes, or session tokens.

9. Email delivery - how to send valid account emails

Because GitHub Pages is only serving static files, email must be sent by the Worker. The most straightforward choice here is Resend, because there is an official Cloudflare Workers tutorial for it and a free plan suitable for low-volume transactional mail.

Recommended email setup
Email provider: Resend.
Sending domain: a real domain or subdomain such as mail.manaforge.example.
Main sender addresses: no-reply@..., support@..., and optionally admin@....
Backend integration point: the Worker calls Resend with an API key stored as a Cloudflare secret.
Tracking table: store an email log in D1 with message type, recipient, related user, send status, and provider response ID.

What “send valid emails” means in practice

- Use a verified sending domain, not a random unverified sender address.
- Publish the DNS records required by the provider: at minimum SPF and DKIM, with DMARC strongly recommended.
- Send from consistent addresses such as no-reply@manaforge.example and support@manaforge.example.
- Send plain-text and HTML content so the message is readable in more clients.
- Keep links short, clear, and tied to one-time tokens or signed records.
- Never send passwords by email.

Email types the system must support from day one

Email type	When it is sent	What the link does	Important rules
Registration received	Immediately after a moderated registration request is accepted by the Worker.	Can open a simple “request received” page or no link at all.	This confirms receipt only. It does not activate the account.
Registration	After an admin	Opens an activation	Token must be one-

Email type	When it is sent	What the link does	Important rules
approved / activation	approves a request.	page where the user creates a password and confirms the email in one flow.	time use and expire, for example in 24 hours.
Registration rejected	Optional, after an admin rejects a request.	Usually no secure link is required.	Keep the message polite and generic. Do not leak internal moderation notes.
Password reset	When a user asks to recover access.	Opens a reset page that accepts a one-time reset token.	Token must be short-lived, for example 30 minutes, and single-use.
Password changed confirmation	After a successful password reset or manual password change.	Usually no action link is required.	This protects the user if the change was not expected.
Email change confirmation	If the app later supports changing the account email address.	Verifies the new email before the switch becomes final.	Treat it like activation: one-time and expiring.
Account suspended notice	Optional, when an administrator disables an account.	Can point to support/help.	Do not expose admin-only details.

Recommended token design

- Generate a long random token on the backend.
- Store only a hash of that token in D1, not the raw token.
- Store the token type, related user, expiration time, and used_at timestamp.
- When the browser returns with the token from the URL, the Worker hashes it again and checks the stored record.
- Once used, mark it used so it cannot be replayed.

Relatable version: treat these tokens like temporary password-reset rows in a database table, not like values the browser can invent or trust on its own.

10. How the email pages work in a static-site setup

The user clicks a link in the email. That link points to a regular GitHub Pages route such as /activate?token=... or /reset-password?token=.... The page itself is just a static frontend screen. The actual trust decision still happens only when that page sends the token to the Worker.

Page the user opens	What the browser shows	What the Worker does
/activate?token=...	A form to set the first password and confirm activation.	Checks token validity, creates password hash, activates the account, burns the token.
/reset-password?token=...	A form to choose a new password.	Checks token validity, stores new password hash, burns the token, returns success.
/request-received	A confirmation screen.	Usually no secure backend action is needed.
/viewer?deck=... or /viewer?share=...	A read-only deck viewer.	Only used for short-link mode when a share slug must be resolved.

11. Proposed database tables in simple language

Table	Purpose	Examples of columns
users	Real accounts that can log in.	id, email, display_name, password_hash, role, status, created_at
registration_requests	Pending and reviewed sign-up requests.	id, email, display_name, note, recaptcha_checked_at, status, reviewed_by, reviewed_at
email_tokens	One-time activation and email-verification tokens.	id, user_id, token_hash, token_type, expires_at, used_at
password_reset_tokens	One-time reset links.	id, user_id, token_hash, expires_at, used_at, requested_ip
sessions	Active login sessions if stored server-side.	id, user_id, created_at, expires_at, revoked_at
projects	User-owned proxy projects.	id, user_id, name, created_at, updated_at
project_cards	Card rows inside a project.	id, project_id, sort_order, card_name, quantity, mana_cost, oracle_text
print_profiles	Saved PDF and page settings.	id, project_id, name, page_size, grid_rows, grid_columns, cut_marks
exports	History of generated PDFs.	id, project_id, generated_at, page_count,

Table	Purpose	Examples of columns
		profile_snapshot_json
share_links	Short-link share payloads.	id, owner_user_id, slug, payload_json, visibility, expires_at
email_log	Audit trail for sent emails.	id, user_id, email_type, recipient, provider_message_id, sent_at, status

12. API endpoints in a more familiar format

Below is the same idea many legacy systems expressed with action parameters, but written as clearer route-based endpoints.

Endpoint	Old mental model	Purpose
POST /api/auth/register-request	submit_request.php	Accept a public registration request after reCAPTCHA verification.
POST /api/auth/login	login.php	Create a logged-in session for an approved account.
POST /api/auth/logout	logout.php	Destroy the current session.
GET /api/auth/me	session_status.php	Return a small summary of the logged-in user.
POST /api/auth/request-password-reset	forgot_password.php	Create and email a reset token.
POST /api/auth/reset-password	reset_password.php	Accept the new password plus reset token.
POST /api/auth/activate	activate_account.php	Accept the activation token and first password.
GET /api/projects	projects.php?action=list	List the user's saved projects.
POST /api/projects	projects.php?action=create	Create a project.
PUT /api/projects/:id	projects.php?action=save&id=...	Update project metadata.
PUT /api/projects/:id/cards	cards.php?action=save	Replace or update project card rows.
POST /api/share-links	share.php?action=create	Create a short-link share record.

Endpoint	Old mental model	Purpose
GET /api/share-links/:slug	share.php?action=view&code=...	Resolve a public shared deck payload.
GET /api/admin/registration-requests	admin_requests.php	List pending and reviewed registration requests.
POST /api/admin/registration-requests/:id/approve	approve_user.php	Approve a request and trigger activation email.

13. PDF generation and print safety rules

The PDF export remains part of the browser-side application. This keeps the backend simpler and avoids building a server-side rendering pipeline for printable sheets.

Mandatory content rules:

- Every generated card face must visibly state PROXY or PLAYTEST.
- The back face must be generic and must not mimic the official MTG card back.
- Do not imitate holo stamps, rarity foil treatment, or any authenticity cues.
- The export screen and the share viewer must show playtest-only notices such as “Not for sale” and “Not tournament legal”.

14. Suggested milestone plan

Milestone	Goal	What should be complete
1. Foundation	Get the stack online.	Next.js export working on GitHub Pages, Worker deployed, D1 connected, Carbon integrated, basic health check screens.
2. Accounts	Get controlled access working.	Registration request form, reCAPTCHA verification, admin approval screen, activation email, login and logout.
3. Recovery and email polish	Make account lifecycle real.	Password reset, password changed notice, email templates, email logging, basic rate limits.
4. Projects and deck editing	Make the core editor usable.	Project CRUD, deck paste/import, saved card rows, first proxy preview.
5. PDF and print profiles	Deliver the main value.	Page setup, PDF generation, saved print profiles, export

Milestone	Goal	What should be complete
		history.
6. Sharing	Deliver deck viewing by URL.	Portable compressed links, short-link fallback, public read-only viewer, share moderation tools.

15. Decisions that keep the project easy to reason about

- Keep GitHub Pages responsible only for serving files, never for trusted business logic.
- Keep the Worker as the single place that knows secrets, permissions, token validity, and database writes.
- Delay anything not required for the first useful version, such as social login, full card API enrichment, or public user profiles.
- Prefer simple, visible flows over “magic” flows. An explicit activation email is easier to explain than hidden automatic account states.
- Treat all share links as read-only views.

16. Appendix A - official documentation links for every chosen system/library

These are the main official references for the stack named in this document.

- **Next.js - static export:** https://nextjs.org/docs/pages/guides/static-exports - How Next.js produces files that can be hosted as a static site.
- **Next.js - deployment overview:** https://nextjs.org/docs/pages/getting-started/deploying - General deployment options and terminology.
- **React:** https://react.dev/ - Core UI library used by Next.js applications.
- **TypeScript:** https://www.typescriptlang.org/docs/ - Language tooling for safer JavaScript development.
- **Carbon Design System:** https://carbondesignsystem.com/ - Design language and component guidance.
- **Carbon React:** https://react.carbondesignsystem.com/?path=/docs/getting-started-welcome--welcome - React component library used in the UI.
- **GitHub Pages:** https://docs.github.com/en/pages - Documentation for static hosting on GitHub.
- **GitHub Pages custom workflows:** https://docs.github.com/en/pages/getting-started-with-github-pages/using-custom-workflows-with-github-pages - Deploying Pages with GitHub Actions workflows.
- **GitHub Actions:** https://docs.github.com/en/actions - Automation and deployment workflows.
- **Cloudflare Workers:** https://developers.cloudflare.com/workers/ - Backend runtime used instead of a traditional PHP host.
- **Cloudflare D1:** https://developers.cloudflare.com/d1/ - Managed SQL database connected to Workers.

- **Cloudflare D1 get started:** https://developers.cloudflare.com/d1/get-started/ - Setup and first-run reference.
- **Wrangler:** https://developers.cloudflare.com/workers/wrangler/ - CLI for developing and deploying Workers and D1 changes.
- **Google reCAPTCHA - choosing versions:** https://developers.google.com/recaptcha/docs/versions - Differences between reCAPTCHA options.
- **Google reCAPTCHA - verify API:** https://developers.google.com/recaptcha/docs/verify - Server-side token verification reference.
- **Resend docs:** https://resend.com/docs - Email API and integration guidance.
- **Resend pricing:** https://resend.com/pricing - Free-plan and paid-plan limits.
- **Cloudflare tutorial - send emails with Resend:** https://developers.cloudflare.com/workers/tutorials/send-emails-with-resend/ - Cloudflare-specific integration example.
- **pdf-lib:** https://pdf-lib.js.org/ - Browser-side PDF generation library.
- **pako:** https://github.com/nodeca/pako - Compression library for portable deck URLs.
- **Node.js docs:** https://nodejs.org/en/docs - Runtime and tooling reference for local development tasks.

17. Appendix B - sample sender addresses and email subjects

Purpose	Suggested sender	Suggested subject line
Registration received	no-reply@manaforge.example	We received your Mana Forge access request
Activation	no-reply@manaforge.example	Activate your Mana Forge account
Password reset	no-reply@manaforge.example	Reset your Mana Forge password
Password changed notice	no-reply@manaforge.example	Your Mana Forge password was changed
Support contact	support@manaforge.example	Mana Forge support update
Administrative account notice	admin@manaforge.example	Mana Forge account status update

18. Final recommendation

Proceed with the stack exactly as planned, but explain it internally using the simpler model: “static files on GitHub Pages, trusted backend on Cloudflare Worker, SQL in D1, email through Resend, bot check through reCAPTCHA.” That description is technically honest and much easier for a PHP-era mental model to follow.

The one structural improvement introduced in this revision is important: moderated registration should lead to an activation email and first-password screen, rather than collecting permanent passwords before approval.